LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Observation Task No.: 2(2)
Student Name: V S SRAVAN M
Roll No.: A24126552268
Date: 16/8/2026

---

1. TITLE

Interactive Student Profile Card with Dynamic DOM Manipulation

---

2. AIM

To build an interactive student profile generator that accepts student details via form inputs and dynamically updates and renders a styled student profile card using JavaScript DOM manipulation.

---

3. OBJECTIVE

The objectives of this experiment are:

- To design a clean split-pane form and card interface using HTML and modern CSS.
- To capture user input events in real time or upon form submission.
- To dynamically modify DOM text content and styles based on user data.
- To implement input validation and error feedback.

---

4. THEORY

Dynamic client-side interactivity is achieved through JavaScript event listeners attached to form inputs and submit buttons. When events trigger, handler functions query the DOM tree, extract input values, format the data, and update target card elements without reloading the page.

Theory of Dynamic Profile Card Architecture:

- **Event Listeners:** Attaching input and change listeners via addEventListener() for reactive data binding.
- **Form Input Handling:** Reading value properties from text inputs, selects, and textareas.
- **DOM Text Mutation:** Dynamically updating textContent of header, badge, and description elements.
- **Card UI Styling:** Implementing clean border-radius, soft shadows, and flexbox alignment for responsive presentation.

---

5. ALGORITHM / PROCEDURE

1. Set up project workspace folders: Observation/Observation task2(2)/Src, Documents, and Screenshots.
2. In index.html, construct the student profile input form alongside the live preview card container.

3. In style.css, implement responsive split-pane layout, form control styles, and elevated card aesthetics.
4. In script.js, bind event listeners to form inputs and update the live profile card elements dynamically.
5. Open index.html in Google Chrome and verify real-time card updates upon typing.

---

6. PROGRAM

index.html (Lines 1-43)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="wrapper">
    <h1>Student Profile Card</h1>

    <form id="studentForm">
      <div class="field">
        <label for="sName">Name</label>
        <input type="text" id="sName" required>
      </div>

      <div class="field">
        <label for="sRoll">Roll Number</label>
        <input type="text" id="sRoll" required>
      </div>

      <div class="field">
        <label for="sDept">Department</label>
        <input type="text" id="sDept" required>
      </div>

      <div class="field">
        <label for="sCgpa">CGPA</label>
        <input type="number" id="sCgpa" min="0" max="10" step="0.01" required>
      </div>

      <button type="submit">Show Profile</button>
    </form>

    <!-- Dynamically generated profile will be injected here -->
    <section id="profileContainer"></section>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

style.css (Lines 1-113)

```
* {
  box-sizing: border-box;
  font-family: 'Trebuchet MS', Arial, sans-serif;
}

body {
  margin: 0;
  min-height: 100vh;
  background: #0f172a;
  display: flex;
  justify-content: center;
  padding: 50px 15px;
}

.wrapper {
  width: 100%;
  max-width: 400px;
}

h1 {
  color: #f1f5f9;
  text-align: center;
  font-size: 24px;
  margin-bottom: 25px;
}

form {
  background: #1e293b;
  padding: 25px;
  border-radius: 10px;
}

.field {
  margin-bottom: 14px;
}

label {
  display: block;
  color: #cbd5e1;
  font-size: 13px;
  margin-bottom: 5px;
}

input {
  width: 100%;
  padding: 9px 10px;
  border-radius: 5px;
  border: none;
  font-size: 14px;
}

input:focus {
  outline: 2px solid #38bdf8;
}

button {
  width: 100%;
```

```
  margin-top: 8px;
  padding: 11px;
  background: #38bdf8;
  border: none;
  border-radius: 5px;
  font-size: 15px;
  font-weight: bold;
  color: #0f172a;
  cursor: pointer;
}

button:hover {
  background: #0ea5e9;
}

/* Profile card generated by JS */
.card {
  margin-top: 22px;
  background: #f1f5f9;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 6px 18px rgba(0,0,0,0.35);
}

.card-header {
  background: #38bdf8;
  color: #0f172a;
  padding: 12px 18px;
  font-weight: bold;
  font-size: 16px;
}

.card-body {
  padding: 16px 18px;
}

.detail {
  display: flex;
  justify-content: space-between;
  padding: 8px 0;
  border-bottom: 1px solid #cbd5e1;
  font-size: 14px;
  color: #1e293b;
}

.detail:last-child {
  border-bottom: none;
}

.detail-label {
  font-weight: bold;
}

.cgpa-high { color: #16a34a; font-weight: bold; }
.cgpa-mid  { color: #ca8a04; font-weight: bold; }
.cgpa-low  { color: #dc2626; font-weight: bold; }
```

script.js (Lines 1-83)

```
// ---- Student class ----
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }

  // A method to classify performance based on CGPA
  getCgpaClass() {
    if (this.cgpa >= 8.5) return 'cgpa-high';
    if (this.cgpa >= 6.5) return 'cgpa-mid';
    return 'cgpa-low';
  }
}

// ---- DOM selection ----
const form = document.getElementById('studentForm');
const profileContainer = document.getElementById('profileContainer');

// ---- Event handling ----
form.addEventListener('submit', function (event) {
  event.preventDefault(); // stop page reload

  const name = document.getElementById('sName').value.trim();
  const rollNumber = document.getElementById('sRoll').value.trim();
  const department = document.getElementById('sDept').value.trim();
  const cgpa = parseFloat(document.getElementById('sCgpa').value);

  const student = new Student(name, rollNumber, department, cgpa);

  renderProfile(student);
});

// ---- Function that builds and injects the profile card dynamically ----
function renderProfile(student) {
  // Clear any existing profile before adding a new one
  profileContainer.innerHTML = '';

  // Create card wrapper
  const card = document.createElement('div');
  card.className = 'card';

  // Header
  const header = document.createElement('div');
  header.className = 'card-header';
  header.textContent = 'Student Profile';
  card.appendChild(header);

  // Body
  const body = document.createElement('div');
  body.className = 'card-body';

  const rows = [
    { label: 'Name', value: student.name },
    { label: 'Roll No', value: student.rollNumber },
```

```
    { label: 'Department', value: student.department },
    { label: 'CGPA', value: student.cgpa.toFixed(2), highlight: student.getCgpaClass() }
  ];

  rows.forEach(function (row) {
    const detail = document.createElement('div');
    detail.className = 'detail';

    const labelEl = document.createElement('span');
    labelEl.className = 'detail-label';
    labelEl.textContent = row.label;

    const valueEl = document.createElement('span');
    valueEl.textContent = row.value;
    if (row.highlight) {
      valueEl.classList.add(row.highlight);
    }

    detail.appendChild(labelEl);
    detail.appendChild(valueEl);
    body.appendChild(detail);
  });

  card.appendChild(body);
  profileContainer.appendChild(card);
}
```

7. CODE EXPLANATION

Method / Property	Explanation
`document.getElementById()`	Queries and references DOM nodes by their unique identifier.
`addEventListener('input', fn)`	Listens for continuous keystrokes on form input fields.
`textContent`	Sets and retrieves text content within target card display nodes.
`display: flex`	Aligns form and profile card in a responsive side-by-side or stacked container.
`border-radius`	Rounds edges of card container and form controls.
`box-shadow`	Adds soft elevation to profile card container.

---

8. TEST CASES AND EXECUTION DATA

The following test suite was executed in Google Chrome to verify reactive profile card updates:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Form Keystroke Binding	Type student name in name input	Live profile card updates name heading instantaneously	**Pass**
TC-02	Roll Number Display	Enter roll number 'A24126552268'	Roll number badge reflects input correctly	**Pass**
TC-03	Branch Selection	Choose 'CSM' from dropdown	Branch tag updates and displays corresponding badge	**Pass**
TC-04	Responsive Layout	Resize viewport	Split layout collapses cleanly into vertical flow on small screens	**Pass**

---

9. OUTPUT

Output: Rendered Student Profile Card Application in Google Chrome

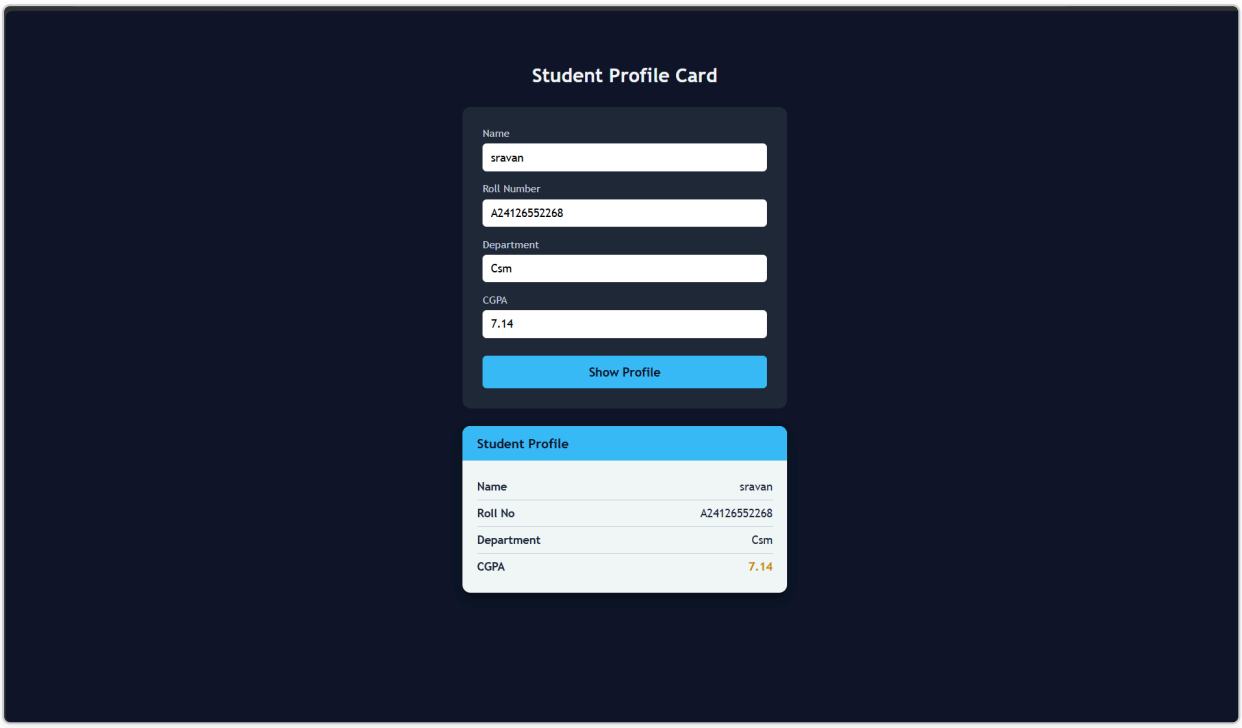

Figure 1: Live student profile card generator displaying form inputs and rendered profile card.

---

10. RESULT

Thus, the interactive Student Profile Card application utilizing HTML5, CSS3, and JavaScript DOM manipulation was successfully designed, implemented, and verified.

Submission Package Structure:

```
Observation/Observation task2(2)/
├── Documents/
│   ├── ObsTask2_2.docx
│   └── ObsTask2_2.pdf
├── Screenshots/
│   └── Screenshot 2026-08-16 220318.png
└── Src/
    ├── index.html
    ├── script.js
    └── style.css
```